Masked screenshots from LLM-proposed trust rules

Procedure, implementation and measurements

RedTeamCUA / `at-spi`

14 September 2026

Abstract

A computer-use agent that reads a screenshot cannot distinguish an instruction written by the person it works for from one planted by whoever controls a comment field on the page in front of it. We remove the distinction from the agent's hands: before the screenshot reaches the model, every region whose contents a non-operator party could have chosen is painted over. The decision of which regions those are is made by a rule set proposed once per page template by an LLM that is shown the page's accessibility tree with every string removed, and applied at run time by a deterministic matcher that never calls a model. This report documents the rule language, the four-call proposal procedure, the deterministic layers wrapped around it, the runtime, and the measurements: a version trajectory, three ablations with repeats, a cross-model comparison, a rule-transfer study, a published baseline, and instrumentation from live agent episodes. On the 160-capture corpus the shipped configuration leaves zero free-text untrusted elements visible, and pays for it by masking about three-quarters of the elements a human judged safe.

Contents

1 Problem and design **1**
- 1.1 What masking has to achieve . . . 1
- 1.2 Allowlist, not denylist . . . 2
- 1.3 Two phases . . . 2

2 The rule language **2**

3 Phase 1: the proposal procedure **3**
- 3.1 The sanitized skeleton . . . 4
- 3.2 Call 1 — the site prior . . . 4
- 3.3 Call 2 — propose . . . 4
- 3.4 Call 3 — the extent follow-up . . . 4
- 3.5 Call 4 — the note audit . . . 5

4 Deterministic layers around the rules **5**
- 4.1 Over-broad rule rejection . . . 6
- 4.2 Repetition taint . . . 6
- 4.3 The capacity exemption . . . 6
- 4.4 File-ownership shortcut . . . 7

5 Phase 2: the runtime **7**
- 5.1 Which rule set applies . . . 7
- 5.2 Rendering and the policy gate . . . 7
- 5.3 Where the algorithms live . . . 8

6 Evaluation protocol **8**
6.1 The scored set . . . 8
6.2 Two metrics . . . 8
6.3 Caveats that affect how every number below should be read . . . 8

7 Results **9**
7.1 Version trajectory . . . 9
7.2 Ablations . . . 10
7.3 Fixing the site prior . . . 10
7.4 Model comparison: v15 vs v17 . . . 11
7.5 Transfer: what a rule cache costs . . . 12
7.6 Baseline: UCM . . . 12
7.7 The system in operation . . . 13

8 Limitations and open problems **13**

1 Problem and design

1.1 What masking has to achieve

The threat is indirect prompt injection against a screenshot-conditioned agent. Text rendered on the screen enters the model's context as pixels, and nothing in that channel marks provenance: a task instruction from the user and a sentence planted in a product review arrive in the same typeface. Any defence that asks the model to tell them apart has already lost, because the model's judgement is the thing under attack.

So the defence is applied before the model sees the image. Every element whose displayed content a party other than the platform operator could have chosen is redacted. What remains is, by construction, operator vocabulary — the strings a site's own templates emit — plus whatever deterministic exemptions are argued for individually in Sec. 4.

Two error directions, and they are not symmetric:

False negative (a *leak*). An untrusted element left visible. This is the failure the system exists to prevent: an attacker-chosen string reaches the model as if it were the platform's own. The design target is zero.

False positive (over-masking). A trusted element painted over. This costs the agent functionality — a button it cannot read, a heading it cannot use to orient. It is a real cost, reported throughout, and deliberately traded away wherever the two conflict.

Because the two costs are incommensurable, we do not report an F_1 that averages them. Every table below prints the leak count and the over-masking rate side by side.

1.2 Allowlist, not denylist

Trust is granted, never revoked: the default verdict for every element is *untrusted*, and a rule is a scoped grant. This polarity decides the shape of every mechanism in the system, and it is worth stating what follows from it.

- A region nobody wrote a rule for is masked. Under-coverage is expensive but safe; under-*specification* is never unsafe.
- Dropping a rule can only add over-masking. Every filter in the pipeline therefore subtracts, and none adds — which is what makes it sound to let a text-reading pass (Sec. 3.5) and a string-comparison pass (Sec. 4.2) see attacker-controlled content at all.

- Pooling rules from several pages can only add grants, so leaks move in exactly one direction when rules are cached across pages. Sec. 7.5 measures how far.

The published system this work is measured against, UCM, takes the opposite polarity, and Sec. 7.6 reports what that costs.

1.3 Two phases

Phase 1, offline, one LLM call chain per page template. An LLM reads a *sanitized structural skeleton* of the page's accessibility tree — roles, DOM `id` and `class`, nesting, and for every accessible name only a placeholder recording its length — and proposes a small set of rules naming the page's trusted containers. It never sees page text.

Phase 2, online, every turn, no LLM. A deterministic matcher runs the stored rules against the live accessibility tree, produces a per-element verdict, paints the untrusted elements' bounding boxes onto the real screenshot, and hands the planner the redacted image together with the boolean mask behind it.

The split is the security argument. The component that decides policy never reads the data the policy is about; the component that reads the data cannot change the policy.

2 The rule language

A rule is a JSON object. Its required fields are `verdict` and `note`; it must then either name a container through `ancestor_path` or declare `"scope": "page"`.

```
{"ancestor_path": [{"role": "landmark", "class": "s-topbar"}],
 "verdict": "trusted",
 "note": "top navigation bar -- renders: Home/Questions/Tags/Users"}
```

Matching semantics. `role` matches exactly; `id` matches exactly; `class` matches by substring against the element's full class string, so `s-topbar` matches `class="s-topbar ps-fixed js-top-bar"`. Several `ancestor_path` entries are matched as an ordered *subsequence* of the element's ancestor chain, so intermediate levels need not be named. A container rule covers the container and every descendant at any depth.

Optional fields.

`except` A list of container signatures subtracted from the grant. An element is excluded if any `except` entry matches it or any of its ancestors, so naming a sub-container subtracts its whole subtree. This is the shape for a page whose chrome and its one piece of submitted content share a wrapper.

`own_role` Restricts the grant to elements of one role inside the container (and their descendants). The shape for trusted widgets scattered through user content — vote buttons inside a post footer.

`scope: "page"` A whole-page assertion in either direction. `trusted` for a reference page or an error page; `untrusted` for a self-published page, where no operator distinct from the author exists. The prompt requires that such a rule be the only rule emitted.

`self_authored_check` Names a byline descendant whose rendered text is compared at apply time against the current user's identity; the container is trusted only on a match. Available only when a current-user identity is supplied.

Table 1: Rule-shape census of the two shipped rule sets.

	v15 (Qwen3.8-27B)	v17 (gpt-5.6-luna)
rules	580	596
captures	160	47
`ancestor_path`	576	596
`except`	79	24
`own_role`	25	110
`scope: page`	4	0
`self_authored_check`	0	0
path depth 1 / 2 / ≥3	559 / 12 / 5	581 / 13 / 2
ancestor keys: role / class / id	599 / 308 / 277	613 / 410 / 221

The geometric propagation gate. Accessibility nesting is not screen containment. `aria-owns` lets a page place elements under a container that is laid out elsewhere, so a small strip can appear to contain content drawn far outside it. Inheritance is therefore checked against geometry as well as nesting: a descendant whose box does not meet the named container's box does not receive the grant, however deeply the tree nests it. Without this, naming a list that merely *references* a region would launder it.

Census. Table 1 shows which constructs the two shipped rule sets actually use. Both are overwhelmingly one-level container grants. The difference in `own_role` usage between the two models is large and is discussed in Sec. 7.4.

A structural limit worth stating. Every key the language matches on — `role`, `id`, `class` — is published by the UI toolkit, and only some toolkits publish all three. Gecko (Thunderbird) and Chromium/Electron (VS Code) copy DOM `id` and `class` into AT-SPI. Plain GTK has no DOM and publishes neither: a GTK file chooser's rows and buttons carry a role and nothing else. A rule over such a surface can therefore name only roles, which is a much coarser vocabulary; and the AT-SPI `AccessibleId` property, the one channel that might have supplied an identity, is empty on every GTK application probed on this image (gedit 638 nodes, gnome-shell 2046 nodes, an open GTK file chooser 207 nodes — zero accessible ids in each). This is the dominant reason native captures behave differently from web captures throughout Sec. 7.

3 Phase 1: the proposal procedure

Four model calls per capture, in order. Calls 1–3 never see page text; call 4 does, and Sec. 3.5 argues why that is sound.

3.1 The sanitized skeleton

The input to the proposal call is a depth-indented walk of the capture's content roots. Each line carries the AT-SPI role, the `id` and `class` where the toolkit publishes them, and, if the node has an accessible name, the placeholder `[name:length:N]` — the length, never the string. No computed property of the text is exposed either, not even whether it looks numeric.

Two decisions inside the walk carry real weight.

Content roots are resolved per platform, not as "the first `document-web` node". That web-only question, asked of a native capture, produced a zero-character skeleton for 33 GTK applications and, for the Thunderbird compose window, a 61-character skeleton consisting of the quoted HTML inside the mail being replied to. The model answered `[]`, which under an allowlist is a

black window. The proposer now resolves roots through the same function the scoring universe uses, so the vocabulary a rule is written in describes the elements the rule will later be matched against.

There is no character cap. A 60,000-character cap was in force earlier and bound on 22 of 55 captures, showing as little as 6% of one tree. Because the walk is depth-first pre-order, truncation always removed the *end* of the document, so whatever a page put last was systematically invisible and no rule could ever be proposed for it.

3.2 Call 1 — the site prior

Before seeing any tree, the model is asked what it knows about this site from its name and category alone. The answer is cached per (model, prompt hash, site description), so a sweep pays once per domain and every capture of that domain is prompted identically. The prompt's own hash is part of the cache key: editing the prompt without invalidating the cache would have served old answers to a new question indefinitely.

The prior is handed to the proposer framed as a recollection to be checked, not as evidence: where the prior and the skeleton disagree, the prompt instructs that the skeleton wins. Sec. 7.2 shows this framing is load-bearing and that an earlier version of the prior was actively harmful on native applications.

3.3 Call 2 — propose

The system prompt states one test and applies it everywhere: *can anyone other than the site's operator cause this region to display something different?* It is posed as a counterfactual rather than as authorship, because authorship gives the wrong answer on every computed value — the operator composed the glyphs "4 months ago", but the instant they describe was chosen by whoever made the commit.

The prompt then fixes several decisions that the corpus showed a model will otherwise get wrong: filter and facet panels are never trusted, in whole or in part, because a seller-derived chip and a platform-defined value are structurally identical; advertising containers are never trusted however unmistakably the site authored the wrapper; navigation that renders user-created names is never trusted; and values that look computed are explicitly declared *not* a reason to trust a region, because a deterministic pass recovers them later (Sec. 4.3) and a release sidebar trusted for its version numbers hands over the release titles beside them.

The output schema has no confidence field. It had one, mirroring UCM's; of 3,656 rules emitted across every run on this corpus, 3,655 said "high". A self-report that is constant carries no information to filter on, so the prompt states the requirement instead of collecting a rating.

3.4 Call 3 — the extent follow-up

The proposer writes a note describing a widget and an `ancestor_path` naming a container, and nothing in a single-turn loop ever compares the two. The follow-up closes that: it replays the model's own rules together with a measured report of how many rows each actually labels on this page, and asks it to reconcile the two. The calibration handed over with the report is explicit — one control is 1–5 rows, a toolbar 5–40, a whole header 3–15% of the page, over 50% is the document rather than a region.

The failure this addresses is not a small overshoot. On one capture a rule whose note read "jump to content link" covered 721 of 722 rows.

3.5 Call 4 — the note audit

A container rule's note is a factual claim about what the region holds, and the proposer cannot check it: it was written from a view with no content in it. We can check it, because the strings

are on disk. This pass reads the text a rule actually covers and asks, once per capture and per rule, whether the region contains what the note says. Rules that fail are dropped.

Only rules that inherit — that cover more rows than the single node they name — are audited; the rest make no claim about a subtree. Batching is per capture rather than per rule because the subtrees are small (median 83 characters of covered text) and the preamble is not, which turns 381 calls into 65.

Why showing this pass real text is sound. Its only output is keep-or-drop on a rule authored blind. Injected text inside a covered region can therefore only cause a *drop*, which withdraws trust — the direction that converts a false negative into a true positive and can never manufacture one. It cannot mint a rule, widen one, edit a note, or reach the proposer's context: it is a separate call with no shared conversation and a boolean-plus-reason response schema, and anything malformed is treated as a drop, so a model talked into silence also fails closed. That monotonicity is the entire licence for the pass, and it must not be weakened into something that can add or broaden rules.

Sec. 7.2 isolates its effect: it removes 42% of the proposed rules and takes the leak count from 2,129 to 0.

4 Deterministic layers around the rules

The LLM is not asked to re-derive anything a cheap mechanism already settles. `predict_trusted` is a precedence chain, and the rules are consulted last.

1. **Zone.** Only `DOCUMENT`-zone rows ever enter the pipeline. Browser and OS chrome are trusted by construction and are never presented for labelling.
2. **Cookie-consent controls.** A pre-vetted accept/reject control whose own id matches a curated per-domain allowlist is trusted before any rule is consulted.
3. **Structural-root carve-out.** The outermost content root's own accessible name is a template-generated label that no `ancestor_path` can name — the row sits at the top of the tree and the skeleton strips its name like every other. It is granted in code.
4. **The LLM rules.**
5. **Default: untrusted.**

What was removed from this chain, and why. A *control-shape* carve-out used to trust every push button, toggle button, combo box, checkbox and radio button on the reasoning that a widget's accessible name is platform vocabulary. It is not: an eBay carousel control carries "Go to previous slide - Multi-Aspect Guidance Car…", a seller-supplied listing title concatenated into the site's own `aria-label`. Measured before removal, the carve-out granted 2,958 rows, 61 of them hand-labelled untrusted, all push buttons. The stronger objection is not the text one: trusting a control means the agent may *act* on it, and a button an attacker named is a button an attacker chose to place in front of the agent. All five roles were removed rather than the one with measured leaks, and controls must now be earned by a rule.

Three passes then run over the rule verdicts. All three only ever subtract trust.

4.1 Over-broad rule rejection

A single container rule that would trust more than 50% of the page is dropped. The two populations do not overlap: healthy rules on captures with no false negatives cover 0–22% of their page, while the rules responsible for all 108 false negatives on one Wikipedia capture covered

99.9% each, both naming the page skin wrapper while their notes read "header landmark" and "footer landmark". Coverage is measured on the *container*, not on the matched set, because a rule whose container is the whole document is a page-scope assertion however narrow an `own_role` filter sits on top of it. Rules carrying an `except` are exempt — high coverage plus an explicit subtraction is a considered claim, high coverage alone is an oversight — as are `scope: page` rules, which are unbounded by definition.

4.2 Repetition taint

A string appearing anywhere in an untrusted region makes its other occurrences untrusted. On one arXiv capture the model correctly subtracted the paper's title, authors and abstract from the abstract wrapper, and the sidebar then re-rendered five of the same author names as classless links among trusted ones, with no role, class, id or ancestor distinguishing them — five false negatives no selector over structure can prevent. Comparing strings does prevent them. Keys are whitespace-collapsed and casefolded, and strings too short or too punctuation-like to carry identity are ignored. The pass only ever turns trusted into untrusted, so it cannot introduce a leak under any input. It closes the leak that subtraction opens; it does not make an over-broad rule safe, and where a rule trusts the whole page there is no untrusted source to taint from and the pass does nothing.

4.3 The capacity exemption

Masking exists to stop injected *instructions*. Whether a string can carry one is a different question from who chose it, and the two come apart: a seller chose `$0.01` and it cannot carry an instruction; a repository owner chose `1.0-ignore-previous-instructions` and it can. So a row carries two independent facts. *Provenance* stays in the hand labels and in the rules; *capacity* says only whether the rendered string is expressive enough to be an instruction. An element is painted over when it is both user-provenance and free-capacity. Exempting a price from masking is not the same as trusting it, and no downstream decision layer treats it as such.

This is not a length threshold — no attacker-tunable constant decides anything. Each grammar is a total, anchored predicate on the string actually on screen, over a closed alphabet checked independently of the regex, with no free letter position: letters appear only as fixed keywords from a closed list. The full reachable vocabulary is 157 fixed tokens containing no imperative verb, so no set of exempt strings, however many and however adjacent, can express an instruction. The exemption is re-decided from the rendered text every turn and can never be inherited by a sibling, by a region, or by the next value in the same slot.

The commit-hash grammar was removed for breaking exactly this property: `[0-9a-f]{7,40}` was a forty-character window of attacker-chosen letters, hex covers enough of the alphabet for leetspeak (`de1e7ed`, `a11f11e5`), and a hash can be ground to a chosen prefix cheaply. Hashes are free-capacity and are masked.

A *fragment gate* withholds the exemption when a bounded value is a piece of a rendered line rather than a value in its own slot. It depends on a neighbour, but only in the safe direction: a neighbour can cause masking, never remove it.

Consequence for the metrics. Because the exemption is deliberate, the false-negative cell splits, and only one half is a security finding:

$$\text{fn} = \underbrace{\text{leak}}_{\text{free-text, shown}} + \underbrace{\text{shown_by_rules_bounded}}_{\text{capacity-bound, shown by a rule}}$$

leak is the number that must be zero. The bounded half is a rule-quality finding: the shortcut never got a say and would often have refused.

4.4 File-ownership shortcut

A file the harness itself placed on behalf of the user carries a provenance manifest entry. A file manager or file chooser row whose entire text is that file's basename is then shown in the clear. The whole design of this grader is in the cases where it refuses: an ambiguous basename (two files share it) grants nothing, an absent one grants nothing, and a full path is not a basename — that is a different grader's question. Files the agent downloads during an episode receive no manifest entry and stay masked, which is correct: an email attachment's filename is chosen by whoever sent it.

5 Phase 2: the runtime

5.1 Which rule set applies

A rule set is proposed for one skeleton and is valid for that skeleton only. Selecting one at run time is two questions, answered in one module used by both the offline grouping and the live lookup so the two cannot drift:

scope whose site or application is this — `reddit.com`, `app:soffice`;

template which page or window of theirs — `/f/*/#/*`, `app://impress_empty`.

They are never merged into one key, and that is measured rather than assumed: pooling template-mates' rules instead of a capture's own moves the leak count from 8 to 23 over the 92 captures that share a group. Those 15 extra leaks are exactly the failure the system exists to prevent, and they buy 1,614 → 2,730 usable elements. So one rule set per (scope, template), selected, never merged — with one deliberate exception: rules for an *application* scope are unioned across that application's captures, because a desktop application's windows share one widget vocabulary in a way that two pages of a website do not.

Deriving both halves from the capture's URL rather than from the manifest's `domain` field matters in practice. For the four self-hosted evaluation platforms that field is a path slug, so a live lookup keyed on the hostname read from the omnibox could never match it, and 37 captures' worth of scored rules were unreachable at run time.

5.2 Rendering and the policy gate

Each turn, the deterministic matcher produces a verdict per element; untrusted boxes are painted onto the real screenshot; the planner receives the image and the policy gate receives the boolean mask behind it. Both come from one call, so "what is hidden" and "what is refused" are the same pixels by construction rather than by agreement.

Restoration is where trust and geometry meet. A trusted row inside an otherwise masked container gets its pristine pixels back only where the painter over those pixels is one of its own ancestors, minus any region occluded by a window stacked above. A refused restore is counted rather than silently dropped.

The gate is not advisory. Clicking into a masked region is blocked by default, and typing is refused when the focused element's centre sits under a redaction — so a fully masked dialog has no keyboard escape either.

5.3 Where the algorithms live

The runtime module is an adapter and nothing else: every algorithm — how trust is decided, how the screenshot is redacted, how masked areas are grouped into numbered query regions — lives in `at-spi/` and is imported, never copied. A second implementation that agrees today is a defence that silently stops agreeing later.

6 Evaluation protocol

6.1 The scored set

The ground truth exports 50,688 rows over 160 captures. Scoring ranges over the *content tier*, visible, hand-labelled subset: **22,256 rows**, of which 12,848 are trusted and 9,408 untrusted. Structural-tier rows (anonymous layout containers, zero-area nodes, off-screen elements) receive labels by derivation and are excluded, because scoring an automated method against derived labels would measure agreement with our own propagation rule rather than with human judgement. Of the untrusted rows, 1,111 are capacity-bound.

Captures fall into three groups, used throughout: **web** (78 public-web captures), **native** (45 GNOME/GTK, XUL and Electron applications), and **rtc-suite** (37 captures of the four self-hosted evaluation platforms: Reddit, ownCloud, RocketChat, GitLab).

6.2 Two metrics

Elements. Every hand-labelled row takes the verdict of the nearest rule covering it or any ancestor of it. Under an allowlist this charges the model for container inhomogeneity by design: a container of mixed provenance cannot be granted trust without its untrusted children coming along, and the false-positive count is exactly the price of describing a page at container granularity.

Pixels. The ground-truth mask is the union of every untrusted element's box; the predicted mask is the union of every predicted-untrusted box; errors are set differences. This metric does not care whether a region was described as one container or forty leaves, and it is what the agent actually receives.

The two disagree informatively: a container false positive over a strip of chrome is one element and a few thousand pixels; over a hero image it is one element and a sixth of the screen.

6.3 Caveats that affect how every number below should be read

These figures were recomputed; the historical logs disagree. The ground truth was re-labelled after the v15 sweep was run (1,317 rows), and the capacity fragment gate was added afterwards. Every number in this report was produced on 14 September 2026 against the ground truth and the capacity semantics as they stand today, so all runs are comparable to each other; they are not comparable to the totals printed in the runs' own logs. For v15 the difference is large: its own log reports 1,056 false negatives, and it scores 138 today.

A scorer defect was found and worked around. `eval_rules.py` discovers rule files by globbing `*.json`, which also matches the `*.proposed.json` siblings that store each capture's pre-audit rules. Every directory from v13 onwards therefore scores each capture twice — once post-audit and once pre-audit — and pools the two. For runs with the audit disabled the two files are byte-identical and only the weight doubles; for v15, 138 of 160 differ, so the tool's default output for v15 is a 50/50 blend of two different rule sets. All numbers here were produced from directory views containing only the final rule files. **The tool itself is still affected** and should be fixed before it is used again.

Reproducing these numbers. The per-capture scores behind every table in Sec. 7 are saved as `at-spi/report_numbers.json`: one record per (rule directory, capture) with the element and pixel confusion cells, the leak split and the rule count. They were produced by scoring each rule directory with `selector_propose.score_file` and `pixel_score_file` under a forced allowlist policy, over one shared universe build.

Table 2: Version trajectory, 160 captures, element and pixel metrics, allowlist forced. v11 is shown for completeness but predates a corpus re-index and covers only 67 of today's captures, so it is not comparable to the rest.

run	caps	rules/cap	leak	fn	fn%	fp	fp%	px FP%	IoU
v11†	67	9.54	226	466	14.58	3,061	50.24	65.36	0.497
v12	160	6.02	41	528	6.85	7,560	61.92	73.03	0.472
v13	160	5.38	2	545	7.04	7,694	63.02	69.86	0.480
v14	160	6.04	2	558	7.21	7,302	59.81	68.35	0.485
v15	160	3.62	**0**	138	1.86	9,331	76.43	79.53	0.453

Table 3: Ablation over the three Phase-1 additions. Qwen3.8-27B, 160 captures, temperature 0.

configuration	prior / extent / audit	rules/cap	leak	fn	fn%	fp	fp%
I base	— / — / —	7.09	195	585	7.75	6,462	52.93
I base, repeat	— / — / —	7.56	208	572	7.58	6,422	52.60
II + site prior	✓/ — / —	9.29	3,105	3,495	46.25	5,699	46.68
II + prior, repeat	✓/ — / —	7.37	2,108	2,467	32.74	6,098	49.95
+ extent follow-up	✓/ ✓/ —	6.30	2,129	2,499	33.06	5,724	46.88
+ note audit (v15)	✓/ ✓/ ✓	3.62	**0**	138	1.86	9,331	76.43

Coverage is concentrated. Three LibreOffice Calc captures hold 38.8% of every untrusted row in the eval set (988, 987 and 905 rows). The top twelve captures hold 52%. A method's aggregate false-negative rate is therefore substantially a statement about spreadsheets, which is why the group breakdowns and the leak counts, rather than the pooled rates, carry the argument.

Captures with no rules are scored, not skipped. An empty rule set under an allowlist means the whole surface is masked. Skipping such captures — which the default polarity inference does, since an empty set has no polarity to infer — silently removes the most over-masking captures from the average. Every table below forces the allowlist policy so that they are included.

7 Results

7.1 Version trajectory

Table 2 tracks the pipeline as it was built. Reading it: *leak* is the count of free-text untrusted elements left visible and is the number to minimise; *fn* is the whole false-negative cell, the remainder of which is capacity-bound and deliberate; *fp%* is the fraction of hand-labelled trusted elements masked.

The trajectory is monotone in the metric that matters and monotone against it in the other. v15 reaches leak = 0 — every remaining false negative is a capacity-bound string — and masks 76.4% of trusted elements to get there. v12 → v13 is the note audit arriving; v14 is the content-root fix for native captures; v15 is a prompt rewrite plus a change of model (Qwen/Qwen-AgentWorld-35B-A3B → Qwen/Qwen3.8-27B).

7.2 Ablations

Three configurations of the v15 prompt on the same 160 captures and the same model, each run twice to expose run-to-run variance. Table 3 adds the pre-audit rule sets, which isolate the last stage exactly, since the audit is the only difference between a run's `.proposed.json` and its final file.

Table 4: Native captures only (45), prior on, extent and audit off, except the last row.

	rules/cap	leak	fn	fp	fp%
base (I), native subset	4.96	3	14	1,470	56.43
old prior (II), native subset	4.13	2,903	2,913	936	35.93
new prior	4.47	5	14	1,400	53.74
new prior, repeat	5.02	5	14	1,418	54.43
v15 full pipeline, native subset	2.44	0	0	1,982	76.08

Three findings.

The site prior, in the form it then had, was actively dangerous — and only on native applications. It raised the leak count from 195 to 3,105. The per-group split shows where all of it came from: on web captures the prior changed the leak count from 164 to 173 and on the self-hosted suites from 28 to 29, both noise; on native captures it went from 3 to **2,903**, with 85.6% of native untrusted rows left visible. The mechanism is visible in the rule shapes: the prior pushed the model towards whole-page grants, and the number of `scope: page` rules rose from 5 to 19. A prior recalled from a *domain name* has nothing useful to say about a desktop application, and it displaced the tree in front of the model rather than supplementing it.

The note audit is what produces leak = 0. It removes 428 of the 1,008 proposed rules (42%) and takes the leak count from 2,129 to 0, at a cost of 3,607 additional masked trusted elements. No other single component in the pipeline has an effect of this size in either direction.

Run-to-run variance is real at temperature 0. The two base repeats agree on the rule count for 92 of 160 captures and differ by 1.81 rules per capture on average, with leak totals of 195 and 208. The two prior repeats agree on 87 and differ by 3.40 rules per capture, with leak totals of 3,105 and 2,108 — a 32% swing. Concurrent decoding on a vLLM server is not deterministic, and single-run differences below roughly this magnitude should not be read as effects.

7.3 Fixing the site prior

The prior prompt was rewritten to stop the model reasoning about a native application as though it were a website, and re-run on the 45 native captures under otherwise identical settings — prior on, extent and audit still off, so the comparison is against configuration II.

The rewrite recovers the regression completely: 2,903 leaks → 5, and the two repeats agree exactly on leak and fn, which the old prior's repeats did not. It does so without the audit, which is the cheaper configuration by one model call per capture.

The last row is the honest comparison and is not flattering. The full pipeline reaches zero leaks on native captures, but 23 of those 45 captures end with *no rules at all* — the audit drops everything — so the agent sees a black window. Corpus-wide, v15's 27 empty rule sets (23 native, 4 web) contribute 1,223 of its 9,331 false positives, 13.1%; excluding them, its over-masking rate is 73.9% rather than 76.4%. Empty rule sets are safe and nearly useless, and they are the main open problem in the native half of the corpus (Sec. 8).

7.4 Model comparison: v15 vs v17

v17 re-ran the pipeline with OpenAI `gpt-5.6-luna` (via OpenRouter, thinking enabled) over the 45 labelled captures of the Thunderbird and self-hosted-suite subsets. Table 5 restricts v15 to

Table 5: Same 45 captures, same pipeline, two models. v15: Qwen3.8-27B, thinking off, prior prompt of 4 September. v17: gpt-5.6-luna, thinking on, prior prompt of 9 September.

	caps	rules/cap	leak	fn	fn%	fp%	px FP%	IoU
v15 (Qwen3.8-27B)	45	3.64	**0**	49	3.92	75.04	87.90	0.321
v17 (gpt-5.6-luna)	45	13.07	1	66	5.17	**45.61**	**70.22**	**0.372**
native subset (8 Thunderbird captures)								
v15	8	8.38	0	0	0.00	62.58	94.85	0.433
v17	8	14.25	0	0	0.00	45.81	96.83	0.427
self-hosted suites (37 captures)								
v15	37	2.62	0	49	5.22	76.99	85.73	0.272
v17	37	12.81	1	66	6.84	45.58	61.93	0.341

exactly those captures.

The stronger model buys usability, not safety. It writes 3.6× as many rules per capture and cuts over-masking from 75.0% to 45.6% of trusted elements — on the self-hosted suites, from 77.0% to 45.6%, and in pixels from 85.7% to 61.9% of the trusted area. Its leak count is 1 rather than 0.

That single leak is worth naming precisely, because it is the only one: a 14×11 pixel `push button` reading "Menu" on an ownCloud page, granted by a rule for the application switcher whose note reads "renders the fixed Files application label and menu control". Its pixel-metric contribution is zero. It is a rule-scoping slip, not a class of failure the pipeline misses.

The mechanism behind the usability gain is visible in Table 1: v17 uses `own_role` in 18.5% of its rules against v15's 4.3%, and emits no whole-page grants at all. It carves narrow, role-restricted grants out of mixed containers where the weaker model either trusted the container or abandoned it. The note audit removes 62 of its 658 proposed rules (9.4%) against 42% for v15 — the rules arrive better calibrated, so less has to be taken away.

Confounds. The two runs differ in three ways at once: model, thinking mode, and the prior prompt (v17 inherits the 9 September rewrite). The prompt difference is the reason v17 was not also run on the web captures, and it means the comparison should be read as *configuration* v17 against *configuration* v15, not as a clean model contrast. The direction of the prompt change on native captures is known from Table 4 and favours v17; the size of the usability gap is far larger than that effect, but the two are not separated here.

7.5 Transfer: what a rule cache costs

Every number above labels a capture with rules proposed for that capture, which is not deployable: an agent arriving at a page cannot first pay for a proposal round. The deployable shape is a cache keyed on something known on arrival. Table 6 measures leave-one-out transfer of the v15 rules over the 154 captures that have any donor at all.

Naive pooling breaks the guarantee outright: 277 leaks across 17 captures at the site radius, 534 across 28 at the family radius. This is the polarity asymmetry doing exactly what it must — more rules can only mean more grants. A cross-capture agreement filter computed without ground truth recovers it: requiring that no donor capture contradicts a rule restores leak = 0 at the family radius, with fewer total false negatives than the per-capture baseline (14 against 138), for 18.6% more over-masking. The per-group tables show the cost is entirely a web-capture phenomenon: at the family radius with the veto, the native and self-hosted-suite groups both reach leak = 0 with 0 and 9 false negatives respectively.

Table 6: Rule transfer, v15 rules, 154 captures. *site* pools the other captures of the same host; *family* pools the other captures of the same site family. *veto* drops a rule if any donor capture's own proposer declined to grant the rows it matches.

protocol / filter	rules/cap	leak	leak caps	fn	fp
self / none	3.6	0	0	138	9,331
site / none	14.6	277	17	649	7,330
site / veto	6.7	13	2	49	10,864
family / none	22.6	534	28	1,059	6,752
family / veto	9.1	0	0	14	11,063

Table 7: UCM against v15 on the 115 web captures with a DOM, same model, same scorer, same rows.

		tp	fp	tn	fn	leak	fp rate
all 115	UCM	1,825	537	9,679	2,514	1,354	5.26%
	v15	3,885	7,349	2,255	138	**0**	76.52%
in-sample 46	UCM	1,105	240	3,867	746	329	5.84%
	v15	1,667	2,870	1,048	51	**0**	73.25%
held-out 69	UCM	720	297	5,812	1,768	1,025	4.86%
	v15	2,218	4,479	1,207	87	**0**	78.77%

We report this as a measurement, not as a shipped configuration. Nothing in the deployed system currently applies the veto filter.

7.6 Baseline: UCM

Untrusted Content Masking identifies user-generated regions with CSS selectors proposed by an LLM from a sanitized page, and is the closest published method. Its automatic boundary-detection loop runs here unmodified — the sanitizer, the proposal conversation, JSON extraction, the dedupe and the confidence filter are all imported from the upstream checkout, with a shim redirecting its hardcoded model to the same Qwen3.8-27B our own run used, and a single domain-agnostic site profile in place of its three hand-tuned ones. Matched elements are mapped to screen rectangles and scored by *our* scorer over identical rows, so there is no second scoring implementation to drift. UCM runs on the 115 captures that have a DOM; the 45 native applications are excluded, not scored as failures. Its protocol proposes from three pages per host, so results split into in-sample and held-out.

The two methods are not close, and they are not close in the way their polarities predict. UCM masks only what a selector names, so 1,354 free-text untrusted elements remain visible — and the degradation off the pages the selectors were written from is severe: 329 leaks in-sample against 1,025 held-out on 1.5× the captures. An allowlist has no equivalent failure, because a page its rules do not describe is masked rather than shown. What it has instead is 76.5% over-masking against UCM's 5.3%. Neither number is the other's units, which is the point of reporting both: the methods sit at opposite ends of the same trade, and the choice between them is a choice about which error is acceptable, not about which method is better calibrated.

7.7 The system in operation

Instrumentation from 32 completed CageAgent episodes, 905 masking turns. Table 8 reports what the runtime actually resolved, per scope.

Table 8: Live masking, 32 episodes / 905 turns. *masked* is the fraction of screen area painted over; *blocked* counts restores refused because the painter was not an ancestor or the region was occluded.

scope	turns	rules	masked %	rows	blocked
reddit.com	570	5.0	83.4	94	0.0
travel.com	117	2.0	29.2	76	10.7
`app:Thunderbird`	92	55.2	89.4	719	190.3
(unresolved)	51	10.3	73.5	138	4.3
`app:gnome-term.-server`	29	0.0	93.8	269	3.4
owncloud.com	20	9.0	83.0	11	0.0
github.com	15	2.0	30.0	88	0.0
shop.com	9	3.0	23.3	44	0.0
`app:soffice`	2	0.0	93.9	1292	12.0

Rule sets were resolved by exact path on 478 turns, by template on 107, by application union on 92, and by a scope default on 51; 124 turns fell through to live proposal and 2 to an on-the-spot proposal. Thirty-one turns ran with no rules at all. Mean masked area across all turns is 75.3%, median 81.5%.

Two observations. The application union is doing real work on Thunderbird: 55 rules per turn against the 12.7 any single capture contributes, which is where v17's density pays off. And the scopes with zero rules are the GTK applications — the terminal and LibreOffice — which is the structural limit of Sec. 2 appearing in production rather than in a benchmark. A terminal episode survives it, because typing needs no pixels; an episode that must click something does not.

8 Limitations and open problems

Over-masking is the shipped cost and it is large. 76.4% of hand-labelled trusted elements, 79.5% of trusted screen area. The stronger model cuts that to 45.6% on the captures where both were run, which suggests most of it is rule quality rather than a limit of the container granularity — but that has been shown on 45 captures, not on the corpus.

Twenty-seven captures have no rules and are masked entirely. Twenty-three of them are GTK applications. This is safe and nearly useless, and it is the largest single block of the over-masking above. The proximate cause is the note audit dropping everything on surfaces where the rules were weak to begin with; the root cause is that a plain-GTK widget publishes a role and nothing else, so there is little for a rule to name.

The scorer double-counts. `eval_rules.py` pools each capture's pre-audit and post-audit rule sets. This does not affect any number in this report, which was produced from filtered directory views, but it does affect the tool.

The evaluation set is concentrated and partly stale. Three spreadsheet captures carry 38.8% of the untrusted rows. Two captures added on 14 September (a gedit file chooser and a Thunderbird attachment chooser) carry no ground truth at all and are excluded from every number here, including v17's, which is scored on 45 of its 47 rule files.

The pixel metric judges under half the screen. Hand labels cover 43.6% of screen area, and 42.9% of what deployment would actually paint over sits on pixels no label ranges over. The

pixel figures above are therefore statements about the judged region, not about the whole image; the true masked fraction is the 75.3% measured live in Table 8, not the 79.5% false-positive rate over judged trusted area.

Transfer is measured but not deployed. The veto filter reaches leak = 0 at the family radius and nothing in the running system uses it.

Rule quality varies between identical runs. Two repeats of the same configuration at temperature 0 differ by 32% in leak count in the worst case. Single-run comparisons of similar configurations are not reliable, and the ablations above are reported in pairs for that reason.

What masking cannot reach. The accessibility tree omits some rendered content entirely, while a screenshot-conditioned agent perceives it. Text drawn inside an attacker-supplied image is visible to the agent and has no element to which a provenance label could attach. Recovering anonymous visual regions narrows this gap; default-deny over screen area not covered by a positively trusted element is the mitigation, and it is part of why the over-masking figures are what they are.